

New chat



how to develop a web ui for ms sql server like ms sql management studio

Building a web UI for Microsoft SQL Server Management Studio-style database management means recreating several categories of functionality:

- Database explorer (schemas, tables, views, procedures)
- SQL editor with syntax highlighting
- Query execution + result grids
- Authentication and connection management
- Table designer / schema browser
- Monitoring and jobs
- Security and permissions
- Import/export tools

A practical approach is to build a lightweight “browser-based SSMS” rather than trying to clone every enterprise feature.

Recommended Architecture

Frontend

Use a modern SPA framework:

Option	Why
React	Best ecosystem for admin dashboards
Vue	Simpler, fast development
Angular	Enterprise-heavy apps

Recommended stack:

- React
- TypeScript
- Monaco Editor (same editor used in VS Code)
- Material UI / Ant Design
- AG Grid or TanStack Table

Useful libraries:

- [Monaco Editor](#)
- [AG Grid](#)
- [React Query](#)

Backend

You need a secure API layer between browser and SQL Server.

Recommended:

Language	Best Framework
Node.js	Express / NestJS
.NET	ASP.NET Core
Python	FastAPI

If targeting SQL Server specifically, ASP.NET Core is excellent because of native Microsoft ecosystem support.

Useful drivers:

- Node: `mssql`
- .NET: `Microsoft.Data.SqlClient`
- Python: `pyodbc`

Official docs:

- [Microsoft.Data.SqlClient](#)
- [Node mssql package](#)

Core Features to Implement

1. Connection Manager

Store:

- server
- database
- username/password
- auth type
- SSL options

Never expose DB credentials directly to frontend.

Recommended:

- encrypted secrets storage
- JWT session tokens
- backend-only DB connectivity

2. Database Tree Explorer

Replicate SSMS object explorer.

API example:

SQL

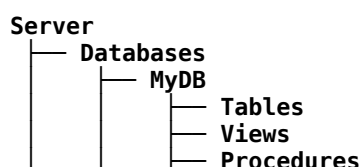
```
SELECT name FROM sys.databases
```

Then:

SQL

```
SELECT * FROM sys.tables
SELECT * FROM sys.views
SELECT * FROM sys.procedures
```

Frontend renders expandable tree:



3. SQL Query Editor

This is the heart of the app.

Use Monaco Editor for:

- SQL syntax highlighting
- autocomplete
- multi-tabs
- error markers
- keyboard shortcuts

You can add IntelliSense by querying metadata:

SQL

```
SELECT TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
```

4. Query Execution Engine

Backend endpoint:

http
POST /execute

Payload:

JSON
{
 "connectionId": "abc",
 "query": "SELECT TOP 100 * FROM Users"
}

Backend:

- validates query
- executes safely
- streams results
- returns rows + metadata

Node example:

JavaScript
const sql = require('mssql');
const result = await pool.request().query(query);

5. Result Grid

Important features:

- pagination
- CSV export
- sorting
- filtering
- virtual scrolling

AG Grid is excellent for this.

6. Schema Designer

Optional but valuable.

Generate:

SQL
CREATE TABLE
ALTER TABLE

from UI forms.

You can introspect schema using:

Printed using [ChatGPT to PDF](#), powered by PDFCrowd [HTML to PDF API](#).

SQL
INFORMATION_SCHEMA.COLUMNS

Security Considerations

This is the most important part.

NEVER:

- run raw SQL directly from browser
- expose SQL Server publicly
- store plaintext credentials
- allow unrestricted admin queries

Recommended protections

Query sandboxing

Restrict dangerous commands:

DROP DATABASE
SHUTDOWN
xp_cmdshell

You can:

- whitelist SELECT-only mode
- role-based permissions
- query parser validation

SQL Injection

Even admin tools should use parameterized queries where possible.

Multi-user Architecture

Recommended model:

```

Browser
  ↓
API Server
  ↓
Connection Pool
  ↓
SQL Server
  
```

NOT:

Browser → SQL Server

Advanced Features (SSMS-like)

Execution Plans

Use:

SQL
`SET SHOWPLAN_XML ON`

Then visualize execution plan graphically.

Live Monitoring

Useful DMVs:

SQL
`sys.dm_exec_requests`
`sys.dm_exec_sessions`
`sys.dm_exec_query_stats`

SQL Formatting

Integrate formatter:

- [sql-formatter](#)
-

Saved Queries / Tabs

Store:

- user workspaces
 - tabs
 - query history
 - favorites
-

Recommended Tech Stack (Best Overall)

Frontend

- React

- TypeScript
- Monaco Editor
- AG Grid
- Tailwind or Material UI

Backend

- ASP.NET Core OR NestJS
- Redis cache
- JWT auth
- WebSockets for live query status

Database

- SQL Server
 - Optional metadata DB for app users/settings
-

Suggested MVP Roadmap

Phase 1

Build:

- login
- DB connection
- query editor
- execute query
- result grid

This alone becomes useful.

Phase 2

Add:

- object explorer
- saved queries
- export/import

- query history
-

Phase 3

Add:

- execution plans
 - monitoring dashboards
 - permissions management
 - schema designer
-

Open Source Projects to Study

These are useful references:

- [Azure Data Studio](#)
- [CloudBeaver](#)
- [Adminer](#)
- [Metabase](#)

CloudBeaver is especially close to a browser-based SSMS.

Recommended Starting Point

If you want the fastest path:

Best combination

Frontend

- React
- Monaco Editor
- AG Grid

Backend

- ASP.NET Core
- Microsoft.Data.SqlClient

This gives:

- best SQL Server compatibility

- enterprise scalability
 - strong authentication support
 - Windows/AD integration possibilities
-

Example High-Level Folder Structure

```
/frontend
  /components
  /pages
  /editor
  /grid

/backend
  /controllers
  /services
  /db
  /auth
```

If You Want a Faster Alternative

Instead of building from scratch:

- fork CloudBeaver
- customize UI/features
- add SQL Server optimizations

This can save months of work.